Rule lists

UNIVERSITÀ DI PISA

Association rule mining

UNIVERSITÀ DI PISA

- Known computational cost
- Simple and flexible

- Local optimization
- No priority: we have rule *sets*

Rule lists

A *rule list* is a sorted set of rules: starting from rule 1, if rule i does not support the given instance, move to rule $i + 1$.

```
(1) if age in (23, 26) and priors in (2, 3) then recidivous
(2) if age in (18, 20) then recidivous
(3) if sex is male and age in (21, 22) then recidivous
(4) if priors > 3 then recidivous
(5) else not_recidivous
```

Example on a recidivism prediction: convicts are judged for early release.

Rule lists

UNIVERSITÀ DI PISA

- Rule priority
- Global optimization
- Compact rules

- Most algorithms work on categorical (often binary) data

Scalable Bayesian Rule Lists, Yang et al.
Learning Certifiably Optimal Rule Lists, Angelino et al.

Adjustments to Rule Lists

Support, confidence, and other measures still hold, but we need to reconsider support: in a rule list, **the first rule to support an instance is the one predicting**.

To adjust, we treat support as an indicator variable, evaluating to 1 for the *first* rule of a given list to apply, and 0 otherwise:

$$supp_A(r, x) = \begin{cases} 1 & if\ r \in A \text{ is the first rule to satisfy } x \\ \\ 0 & otherwise \end{cases}$$

Adjustments to Rule Lists

UNIVERSITÀ DI PISA

```
(1) if age in (23, 26) and priors in (2, 3) then recidivous
(2) if age in (18, 20) then recidivous
(3) if sex is male and age in (20, 22) then recidivous
(4) if priors > 3 then recidivous
(5) else not_recidivous
```

age	**priors**	**sex**	r_1	r_2	r_3	r_4	r_4
24	4	m					
20	1	f					
20	5	m					

What is the value of the support indicator variable?

Adjustments to Rule Lists

UNIVERSITÀ DI PISA

```
(1) if age in (23, 26) and priors in (2, 3) then recidivous
(2) if age in (18, 20) then recidivous
(3) if sex is male and age in (20, 22) then recidivous
(4) if priors > 3 then recidivous
(5) else not_recidivous
```

age	**priors**	**sex**	r_1	r_2	r_3	r_4	r_4
24	4	m	1	0	0	0	0
20	1	f	0	1	0	0	0
20	5	m	0	1	0	0	0

CORELS: Certifiably Optimal Rule Lists

Try it online! *https://corels.cs.ubc.ca/corels/*

Branch and bound algorithms

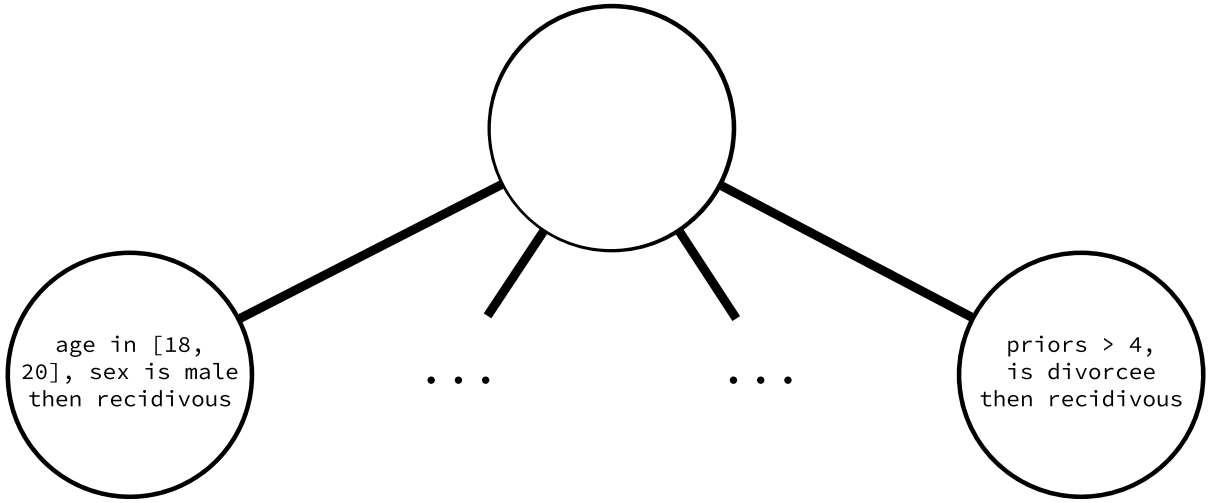
Family of optimization algorithms that defines a space of solutions to search, according to a given objective function. Exploring the whole space is unfeasible, hence branch and bound algorithms define:

- A **set of feasible solutions** to explore from a starting set: the *branches* of the solution tree to explore
- A **bound** for the objective: it allows to *prune* branches, thus reducing the search space

Why B&B? The search tree of rules

Università di Pisa

A search tree has an exponentially large number of states! For a tree of order k and depth d, $\mathcal{O}(k^d)$ states!



A branch and bound tree for rule lists, trimmed at depth 1.

Size of the rule space: full-length rule

Università di Pisa

For simplicity, assume m features, each discretized in k bins, and each feature can only appear in a rule once. A rule of length m can be constructed with a carthesian product of predicates from each of the m k-large sets: we have k^m combinations!

age	salary	assets
`[0-18]`	`[0, 0]`	`[0, 0]`
`[19, 21]`	`[1, 1000]`	`[1, 100]`
`...`	`...`	`...`

Note: we are "only" considering the premises: the class of the rule would add another multiplicative factor!

Size of the rule space: all rules

Università di Pisa

Considering shorter rules, the same applies. We consider $m - i$ sets (features) sampled from m sets, for $\binom{m}{m-i}$ combinations, and k^{m-i} values: a search space of $\mathcal{O}(\binom{m}{m-i} k^{m-1})$ size! Summing up over all lengths, we have

$$\sum_{i=1}^{m-1} \binom{m}{m-i} k^{m-i} + k^m$$

possible rules. Note: to construct the space of rule lists, we have to consider all possible permutations of such rules!

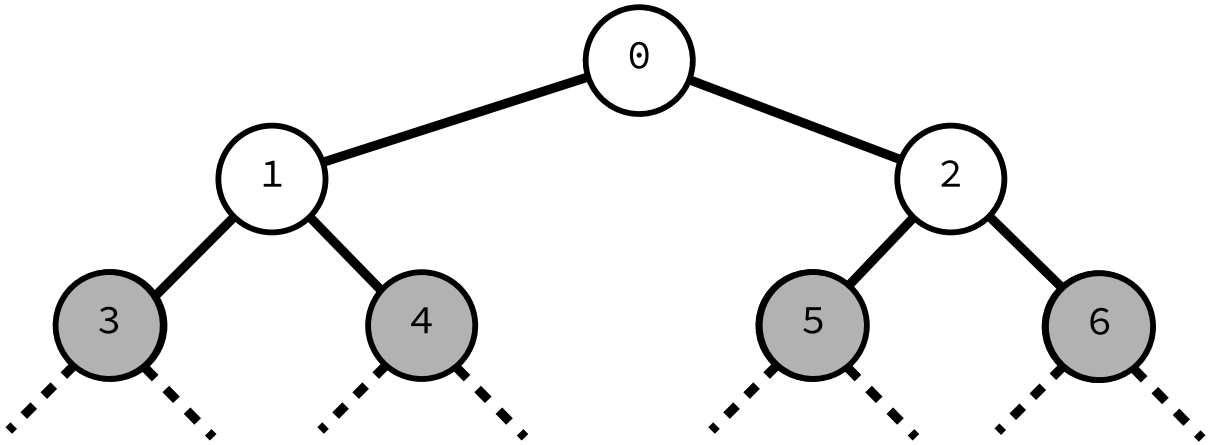
Note: we are "only" considering the premises: the class of the rule would add another multiplicative factor!

Branch and bound algorithms

Università di Pisa

Exploration populates a queue of states to consider: the larger the queue, the larger the computational cost. A search algorithm simply

- Inserts states in the queue
- Pops states from the queue



A branch and bound tree, a set of states to explore (in grey). The associated queue includes states $3, 4, 5$, and 6. Insertition order in the queue depends on the search strategy.

Certifiably Optimal Rule Lists: CORELS

UNIVERSITÀ DI PISA

Modeling

First we model the rule list as a tuple $R : (P_R, Y_R, y_R^0)$ of

- $P_R : (p^1, \ldots, p^{k^R})$ premises of the rules, i.e., their body or antecedents
- $Y_R : (y_R^1, \ldots, y_R^{k^R})$ labels associated to each rule
- y_R^0 default label: applied when no rule is satisfied

An ordering is defined among rules: if a rule list R is a prefix of a rule R^+, then $R \preceq R^+$. That is, nodes in the search tree extend their parents by adding a rule to their lists.

Example

Università di Pisa

```
(1) if age in (23, 26) and priors in (2, 3) then recidivous
(2) if age in (18, 20) then recidivous
(3) else not_recidivous
```

- k^R is `2`
- $P_R : (p^1, \ldots, p^{k^R})$ is `(age in (23, 26) and priors in (2, 3), age in (18, 20))`
- $Y_R : (y_R^1, \ldots, y_R^{k^R})$ is `(recidivous, not_recidivous)`
- y_R^0 is `not_recidivous`

Note: k^R only counts non-default rules.

CORELS: objective

In a rule list, the empirical error of the model on a labelled dataset (X, Y) is given by

$$l(R, X, Y) = \underbrace{l_R(R, X, Y)}_{supported} + \underbrace{l_0(R, X, Y)}_{not\ supported},$$

where:

- $l_R(R, X, Y) = \frac{1}{n} \sum_{x,y \in X \times Y} \sum_{j=1}^{k^R} supp_{P_R}(p_R^j, x) \mathbb{1}[y \neq y_R^j]$
- $l_0(R, X, Y) = \frac{1}{n} \sum_{x,y \in X \times Y} \sum_{j=1}^{k^R} \underbrace{(1 - supp(P_R, x))}_{\text{not supported by any rule}} \mathbb{1}[y \neq y_R^{k^R}]$

CORELS: loss and lower bound

We regularize complexity of the model by weighing in its size, to define the loss L of the model:

$$L(R, X, Y) = \underbrace{l(R, X, Y)}_{error} + \underbrace{\lambda k^R}_{regularization}$$

We can now define lower bound $b(\cdot, \cdot, \cdot)$ of the loss, which only considers instances which are supported by non-default rules:

$$b(R, X, Y) = l_R(R, X, Y) + \lambda k^R \leq L(R, X, Y).$$

Trimming the space

UNIVERSITÀ DI PISA

Trimming the search tree considers up to three components:

- The current best model R^C, and its loss L^C
- An optimal list R^*, and its loss L^*
- The bounds associated to both

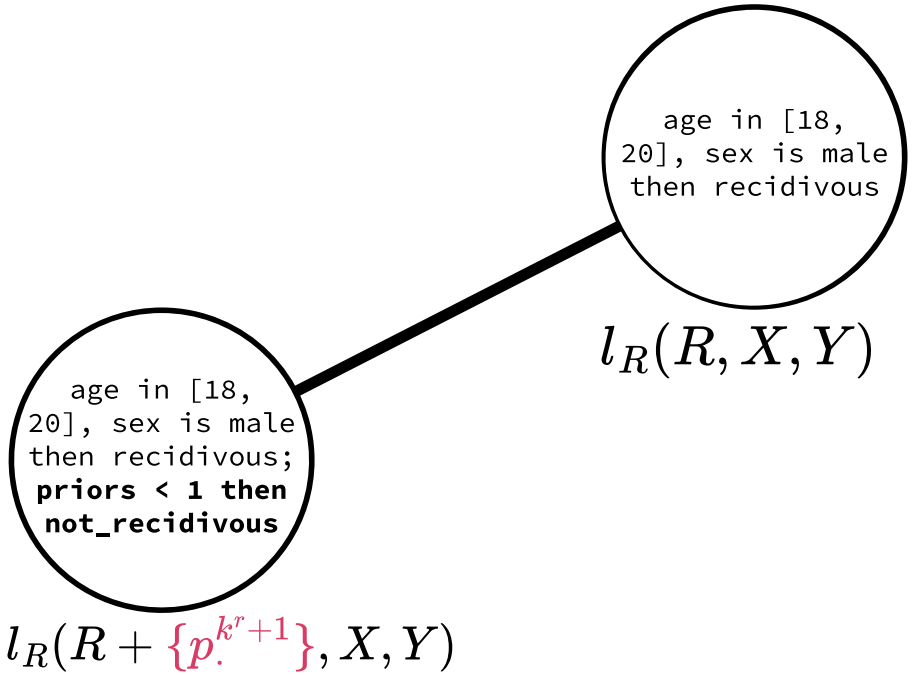
CORELS: bound on loss

UNIVERSITÀ DI PISA

For two rule lists R, R^+ s.t. $R \preceq R^+$

$$b(R, X, Y) \leq L(R^+, X, Y).$$

By definition of l_R, adding rules to the list can only grow l_R. Inductively, for **any prefix with a bound higher than desired we can trim all of its suffixes**!



Rule lists, their prefix relationship, and the relationship with the loss and bound.

CORELS: bound on loss

$$l_R(R^+, X, Y) = \frac{1}{n} \sum_{x,y \in X \times Y} \sum_{j=1}^{k^{R^+}} supp_{P_R}(p^j, x) \mathbb{1}[y = y_R^j]$$

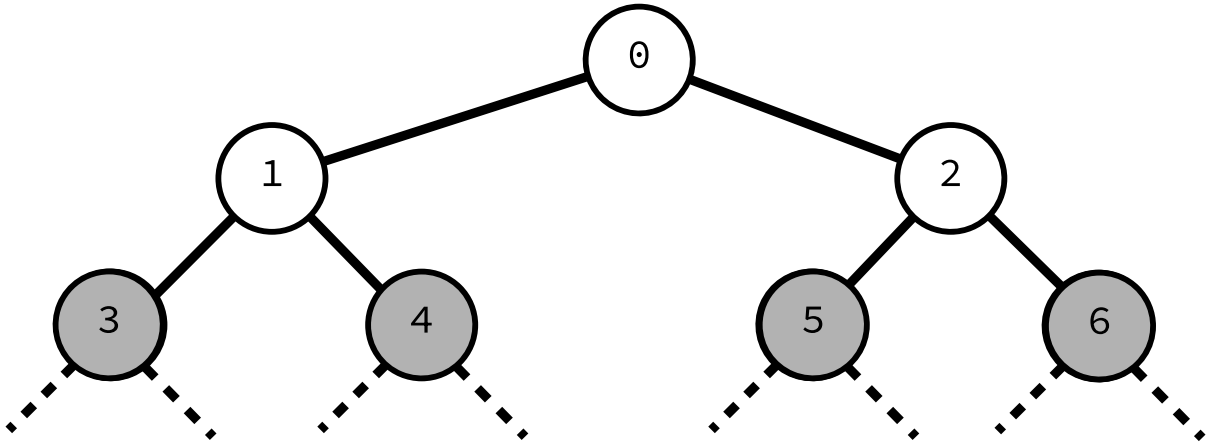
$$= \frac{1}{n} \sum_{x,y \in X \times Y} (\sum_{j=1}^{k^R} supp_{P_R}(p^j, x) \mathbb{1}[y = y_R^j] + \sum_{j=k^R}^{k^{R^+}} supp_{P_R}(p^j, x) \mathbb{1}[y = y_R^j])$$

$$= \underbrace{\frac{1}{n} \sum_{x,y \in X \times Y} (\sum_{j=1}^{k^R} supp_{P_R}(p^j, x) \mathbb{1}[y = y_R^j])}_{l_R} + \underbrace{\frac{1}{n} \sum_{x,y \in X \times Y} \sum_{j=k^R}^{k^{R^+}} supp_{P_R}(p^j, x) \mathbb{1}[y = y_R^j])}_{\geq 0}$$

CORELS: bound on loss

Università di Pisa

Given a current best bound b^C, I can trim all states in the queue with bound $> b^C$.



A branch and bound tree. States to explore highlighted in grey.

CORELS: bound on length

UNIVERSITÀ DI PISA

For a maximum of $\bar{r}$ antecedents, the lengths k^{R^*}, k^{R^c} of the optimal (R^*) and current (R^c) best rule list are related by

$$k^{R^*} \leq \min(\frac{L^c}{\lambda}, \bar{r}).$$

Why?

- $k^{R^*} \leq \bar{r}$ is trivial (at most $\bar{r}$ rules available)
- For $\frac{L^c}{\lambda}$

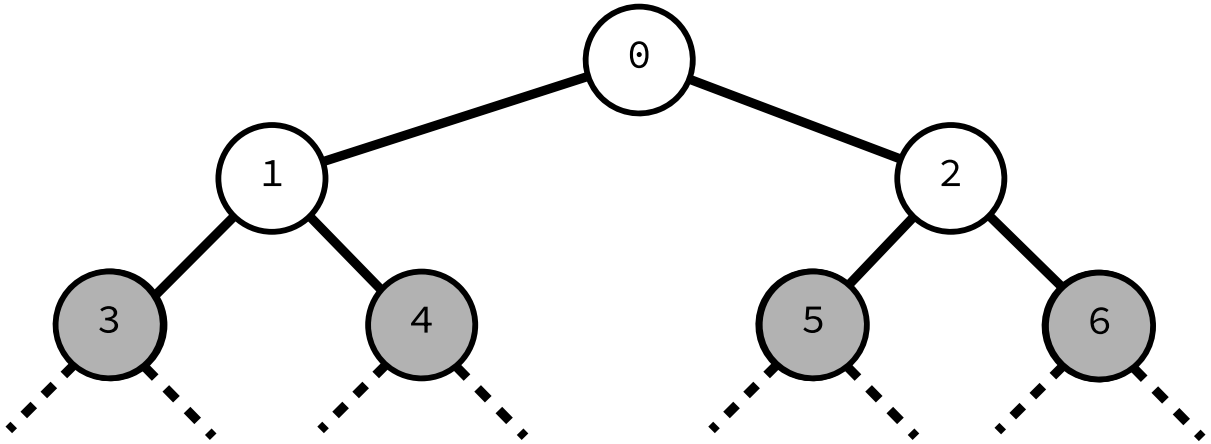
$$k^{R^*} \leq \frac{L^c}{\lambda}$$

$$\underbrace{\lambda k^{R^*}}_{\leq L^*} \leq \underbrace{L^c}_{\geq L^*}$$

CORELS: bound on length

Università di Pisa

Given a current best loss b^C,
I can trim all states at depth
$> \max(\frac{L^c}{\lambda}, \bar{r})$.



A branch and bound tree. States to explore highlighted in grey.

CORELS: bounds on rule accuracy

We can further trim the branched states by bounding the number of correctly classified instances by any rule. In other words, for an optimal rule list, what is the minimum accuracy α^p a new rule p should have to be added?

We define α^p and the related loss l^p as

$$\alpha^p \equiv \frac{1}{n} \sum_{i=1}^{n} supp(p, x^i) \mathbb{1}[y_i = y^p], \qquad l^p \equiv \frac{1}{n} \sum_{i=1}^{n} supp(p, x^i) \mathbb{1}[y^i \neq y^p].$$

CORELS: bounds on rule accuracy

In the worst-case scenario, all instances covered by p are instead covered by other rules and misclassified, thus the difference is bounded by p's accuracy

$$l(R, X, Y) - l(R + [p], X, Y) \leq \alpha^p.$$

We can directly relate this to the regularization hyperparameter λ!

CORELS: bounds on rule accuracy

Università di Pisa

$$
\begin{aligned}
l(R,X,Y) - l(R+[p],X,Y) &\leq \alpha^i \\
l(R,X,Y) &\leq \alpha^i + l(R+[p],X,Y) && \text{move to RHS} \\
l(R,X,Y) + \lambda k^R &\leq \alpha^i + l(R+[p],X,Y) + \lambda k^R && \text{add } \lambda k^R \text{ to both sides} \\
L(R,X,Y) &\leq \alpha^i + L(R+[p],X,Y) - \lambda && \text{by definition of } L, \text{ and } k^{R+[p]} = k^R - 1 \\
-\alpha_i + L(R,X,Y) &\leq L(R+[p],X,Y) - \lambda && \text{move to LHS} \\
\alpha_i - L(R,X,Y) &\geq -L(R+[p],X,Y) + \lambda && \text{sign flip}
\end{aligned}
$$

Note that $L(R,X,Y) \leq L(R+[p],X,Y)$, thus it must be $\alpha_i \geq \lambda$: **any rule added to the list must have accuracy higher than λ!**

CORELS: three bounds

Loss increase

More rules, higher loss.

$$b(R, X, Y) \leq L(R^+, X, Y)$$

List length

Lists longer than

$$\min(\frac{R^c}{\lambda}, \bar{r})$$

can be ignored.

Rule accuracy

A rule p should be added if and only if it has accuracy $\alpha^p \geq \lambda$.

Branch and bound in CORELS

UNIVERSITÀ DI PISA

```
queue = [()]
current_best, current_best_state = +inf, None
while len(queue) > 0:
        state = queue.pop()  # get state
        if bound(state, data, labels) < current_best:
                state_loss = L(state, data, labels)
                # update if new best is found
                if state_loss < current_best:
                        current_best = state_loss
                        current_best_state = state

                # branch: trim() uses the bounds to trim suboptimal states
                queue += trim(state.generate_children(), data, labels, state)  #


return current
```

References

- *Learning Certifiably Optimal Rule Lists for Categorical Data*, Elaine A., Larus-Stone N., Alabi D., Seltzer M, Rudin C. *Journal of Machine Learning Research* (Journal version)
- *Learning Certifiably Optimal Rule Lists for Categorical Data*, Elaine A., Larus-Stone N., Alabi D., Seltzer M, Rudin C. *SIGKDD 2017* (Conference version)

Note: only the bounds in the slides are part of the program.